

VITyarthi Project Report

STUDENT PERFORMANCE ANALYSIS SYSTEM

Using Python and Fundamental Algorithms

Submitted by

Rishit Singh

Registration No.: 26BAI10405

Course: Introduction to Problem Solving and Programming

Course Code: CSE1021

Faculty: M DARSHAN TEJA

Semester: Fall Sem 2026-27

Academic Year 2026–27

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots and Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. Conclusion
16. References

1. Introduction

Student performance records contain useful information such as marks, attendance, grades, and subject-wise performance. When these records are handled manually, basic tasks such as searching, updating, calculating averages, and preparing reports can become repetitive and error-prone.

The Student Performance Analysis System is a command-line Python application developed to manage student records and perform basic academic analysis. The project applies fundamental programming concepts and algorithms such as functions, modules, loops, conditional statements, summation, counting, maximum and minimum finding, linear search, duplicate removal, and partitioning.

The system follows a modular design so that student management, validation, algorithms, analysis, reports, sample data, and testing are separated into individual Python modules.

2. Problem Statement

Educational users may need a simple way to maintain student performance information and obtain useful summaries without manually calculating every result. The problem addressed by this project is to design and implement a small, reliable command-line system that can manage student records and analyze academic performance using fundamental Python programming concepts.

The current system stores student information in memory using Python data structures. It is intended as an academic project demonstrating algorithmic problem solving rather than as a production student information system.

3. Objectives

- Design a modular command-line application using top-down problem solving.
- Implement CRUD operations for student records.
- Validate user input for student IDs, names, marks, and attendance.
- Apply fundamental algorithms to real-world student performance data.
- Calculate individual and class-level performance statistics.
- Generate readable individual, class, and subject reports.
- Verify important functions using automated test cases.
- Demonstrate the use of Python lists, dictionaries, sets, functions, and modules.

4. Functional Requirements

ID	Function	Description
FR1	Add Student	Create a new student record after validating the input.
FR2	View Students	Display all stored student records.
FR3	Search Student	Find a student using the student ID.
FR4	Update Student	Modify student information and marks.
FR5	Delete Student	Remove a student record after confirmation.

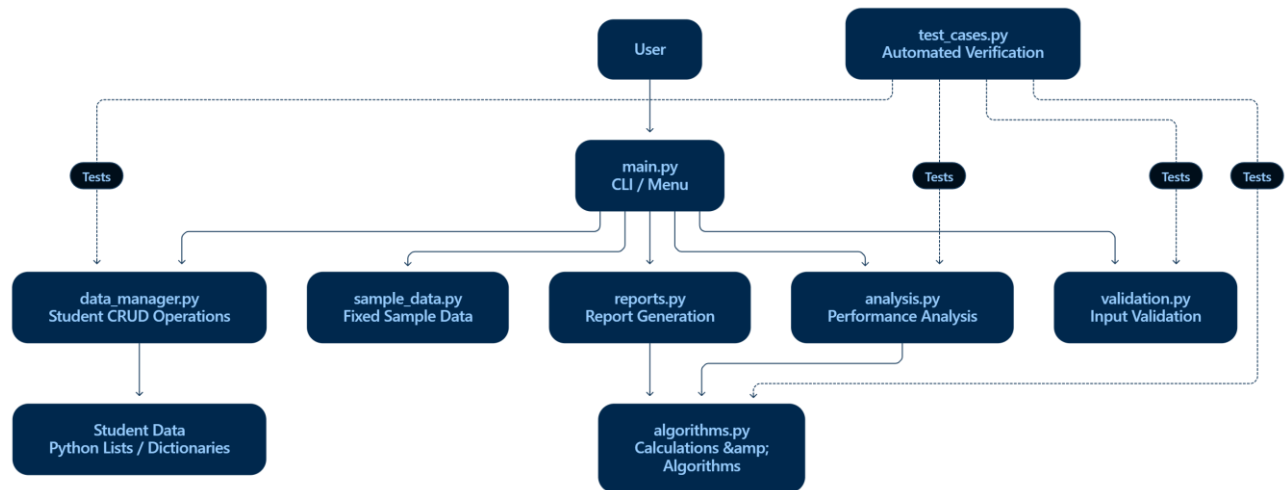
FR6	Class Analysis	Calculate class averages, pass/fail counts, and performance categories.
FR7	Individual Report	Generate a detailed report for one student.
FR8	Class Report	Generate a summary report for the class.
FR9	Subject Report	Calculate subject-wise performance statistics.
FR10	Sample Data	Load a fixed set of eight student records for demonstration.
FR11	Testing	Run automated tests for selected algorithms and calculations.

5. Non-Functional Requirements

Requirement	Description
Usability	The system should provide a clear menu-driven command-line interface.
Input Validation	Invalid IDs, names, marks, attendance values, and duplicate IDs should be handled.
Maintainability	Separate modules should make individual functions easier to modify and test.
Performance	The project uses simple algorithms appropriate for a small in-memory dataset.
Reliability	Important calculations and algorithms are verified using test cases.
Portability	The project should run from a terminal on systems with Python 3 installed.

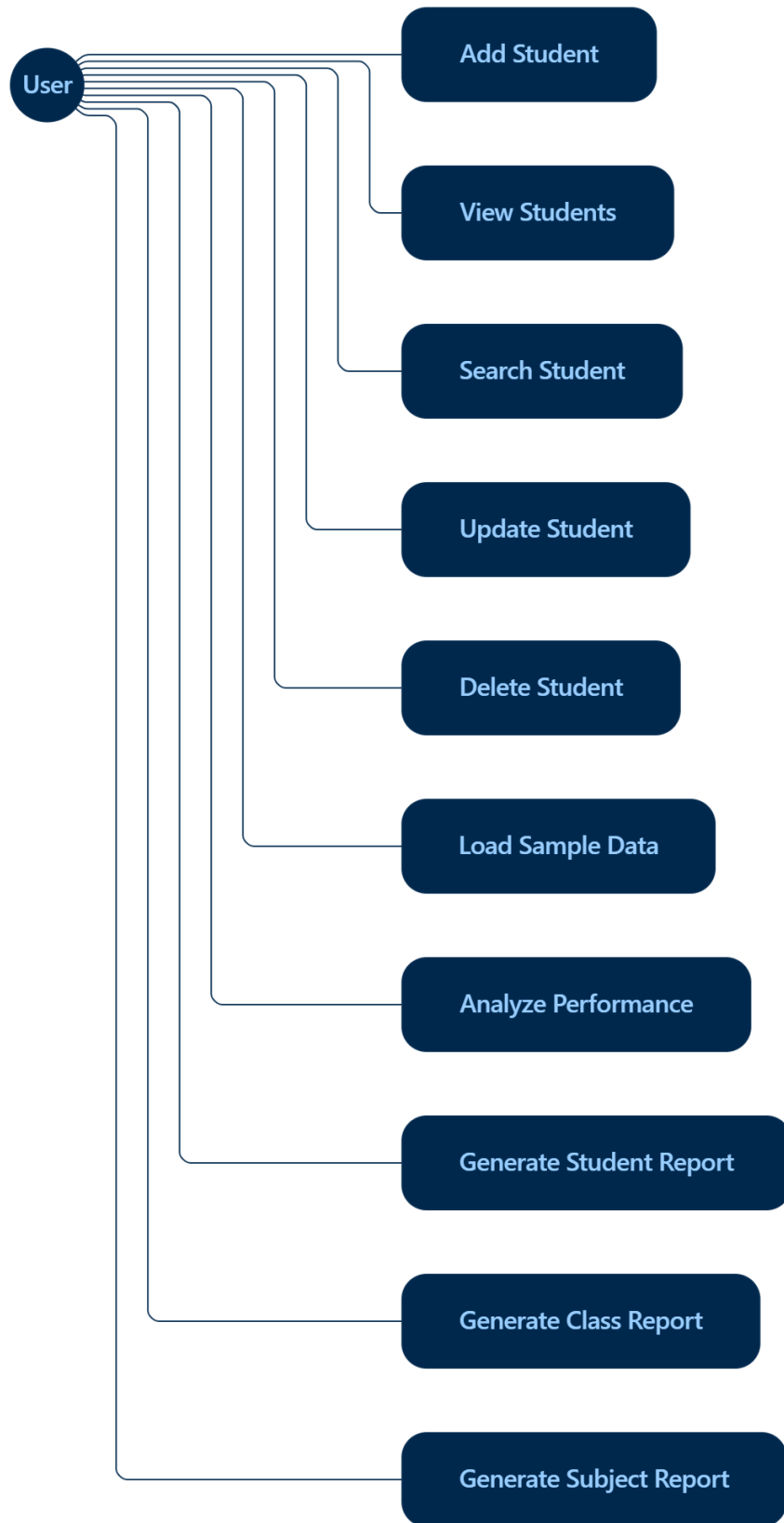
6. System Architecture

The project uses a modular architecture based on top-down design. The main program controls the menu and calls functions from supporting modules. Data management handles CRUD operations, validation checks user input, algorithms provides reusable fundamental algorithms, analysis calculates performance statistics, reports formats results, sample_data supplies fixed demonstration records, and test_cases verifies selected functions.

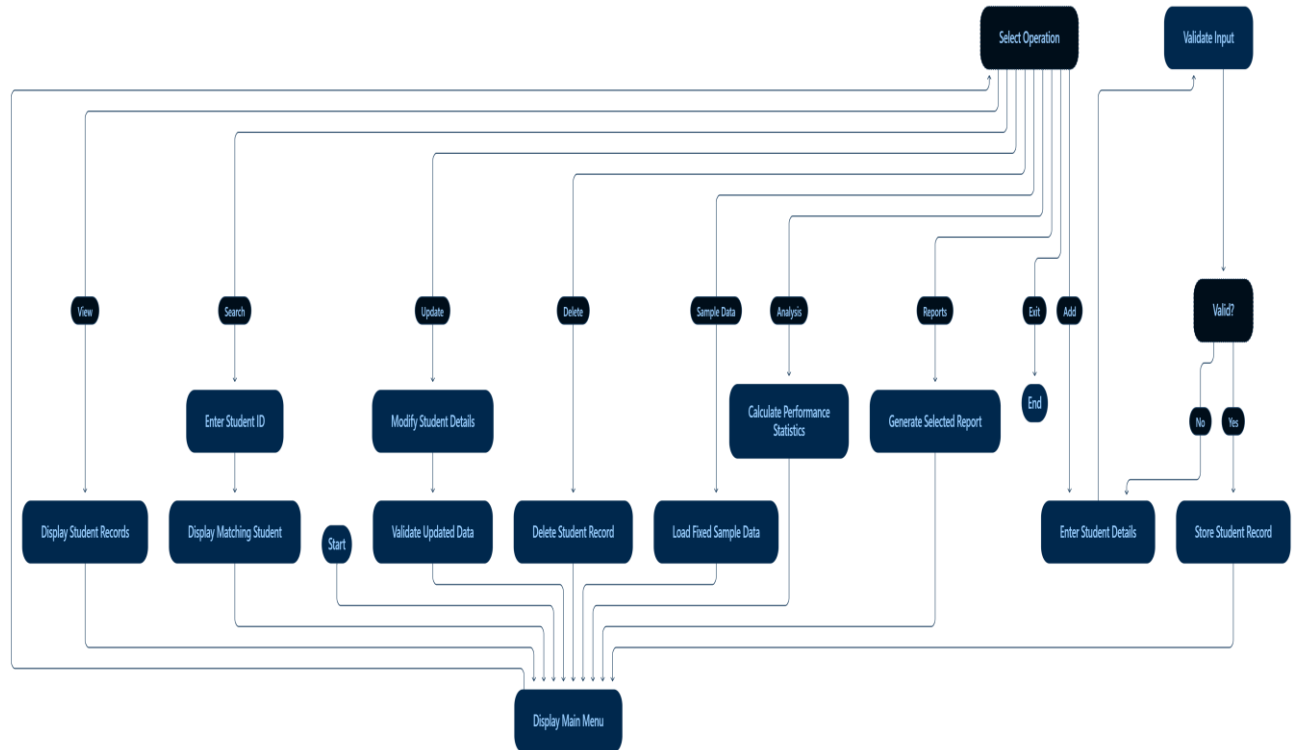


7. Design Diagrams

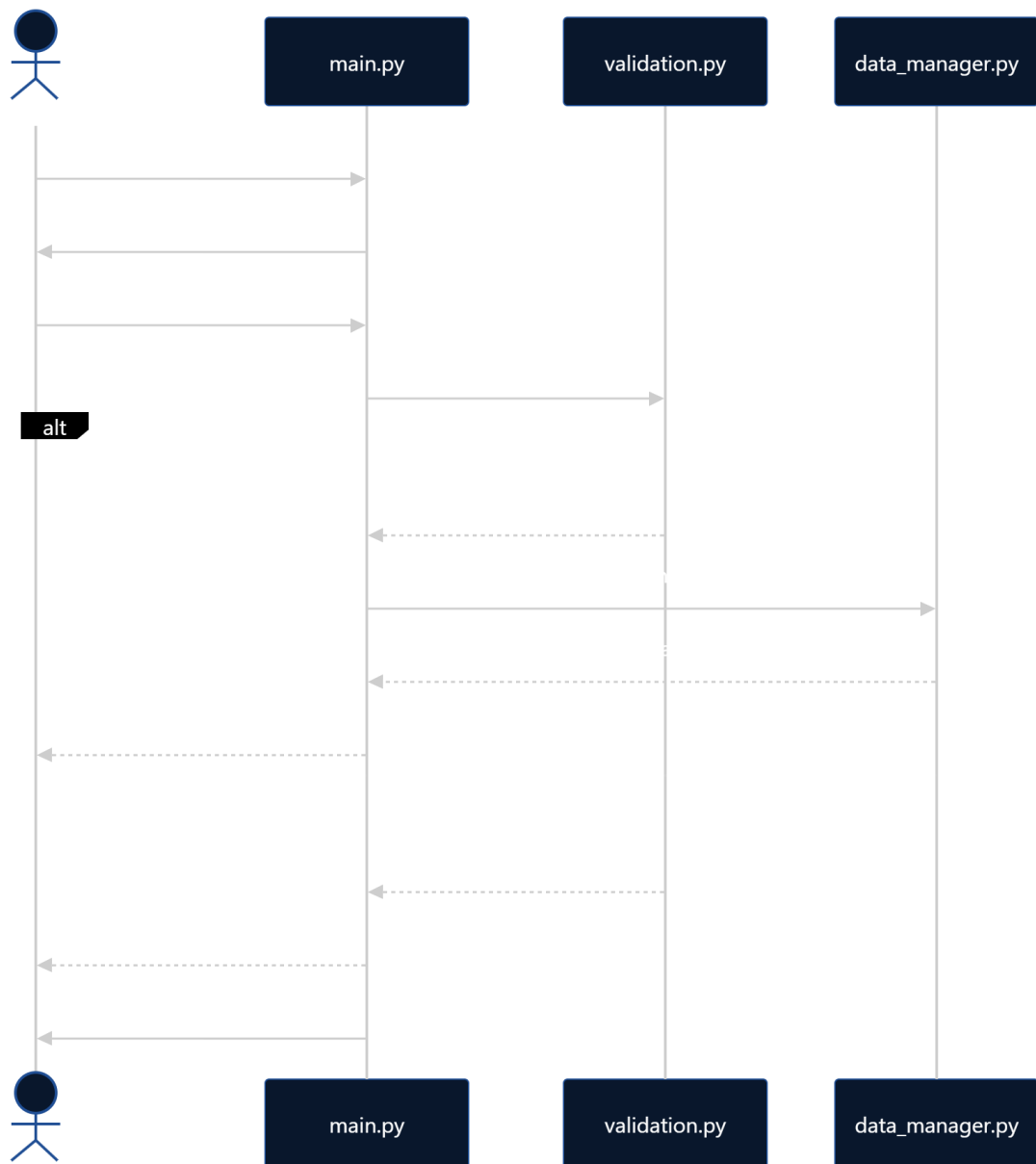
7.1 Use Case Diagram



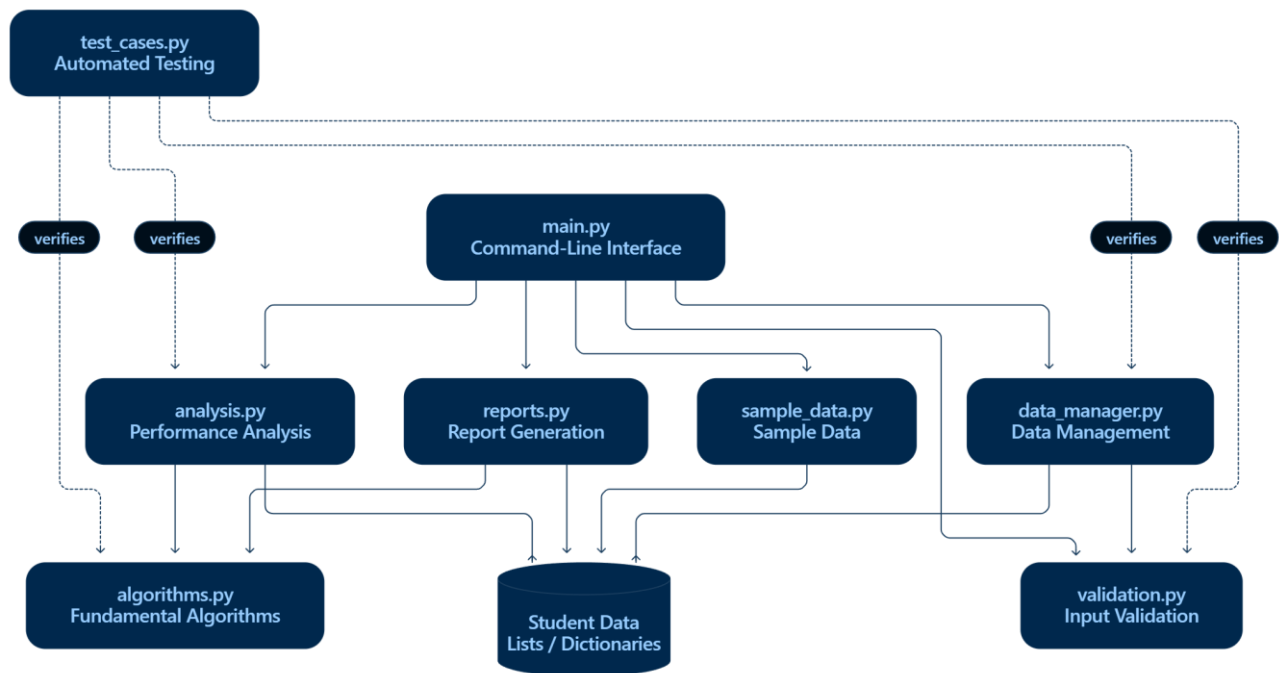
7.2 Workflow Diagram



7.3 Sequence Diagram



7.4 Class/Component Diagram



8. Design Decisions and Rationale

Decision	Rationale
Modular design	Separate Python files were used for validation, algorithms, data management, analysis, reports, sample data, and testing. This supports maintainability and top-down design.
In-memory data	Lists and dictionaries are sufficient for the small academic dataset and avoid unnecessary database complexity.
Linear search	Linear search is simple to implement and suitable for a small number of student records.
Fixed sample data	Fixed data gives repeatable results during demonstrations and testing.
Command-line interface	A terminal interface satisfies the requirement for command-line execution and keeps the project focused on programming fundamentals.

9. Implementation Details

Module	Responsibility
main.py	Controls the menu, program flow, sample-data loading, and interaction between modules.
data_manager.py	Implements student CRUD operations and student display/search functionality.
validation.py	Validates student IDs, names, marks, and attendance.
algorithms.py	Contains summation, average, maximum, minimum, counting, linear search, duplicate removal, and partitioning functions.
analysis.py	Calculates totals, averages, percentages, grades, pass/fail status, class statistics, and student categories.
reports.py	Generates individual, class, and subject reports.
sample_data.py	Provides eight fixed sample student records.
test_cases.py	Verifies selected functions including summation, average, maximum, minimum, duplicate removal, grading, and linear search.

9.1 Data Structures Used

- List: stores the collection of student records.
- Dictionary: represents each student and stores fields such as ID, name, marks, and attendance.
- Nested dictionary: stores subject-wise marks for each student.
- Set: used for duplicate removal in the algorithm demonstration.

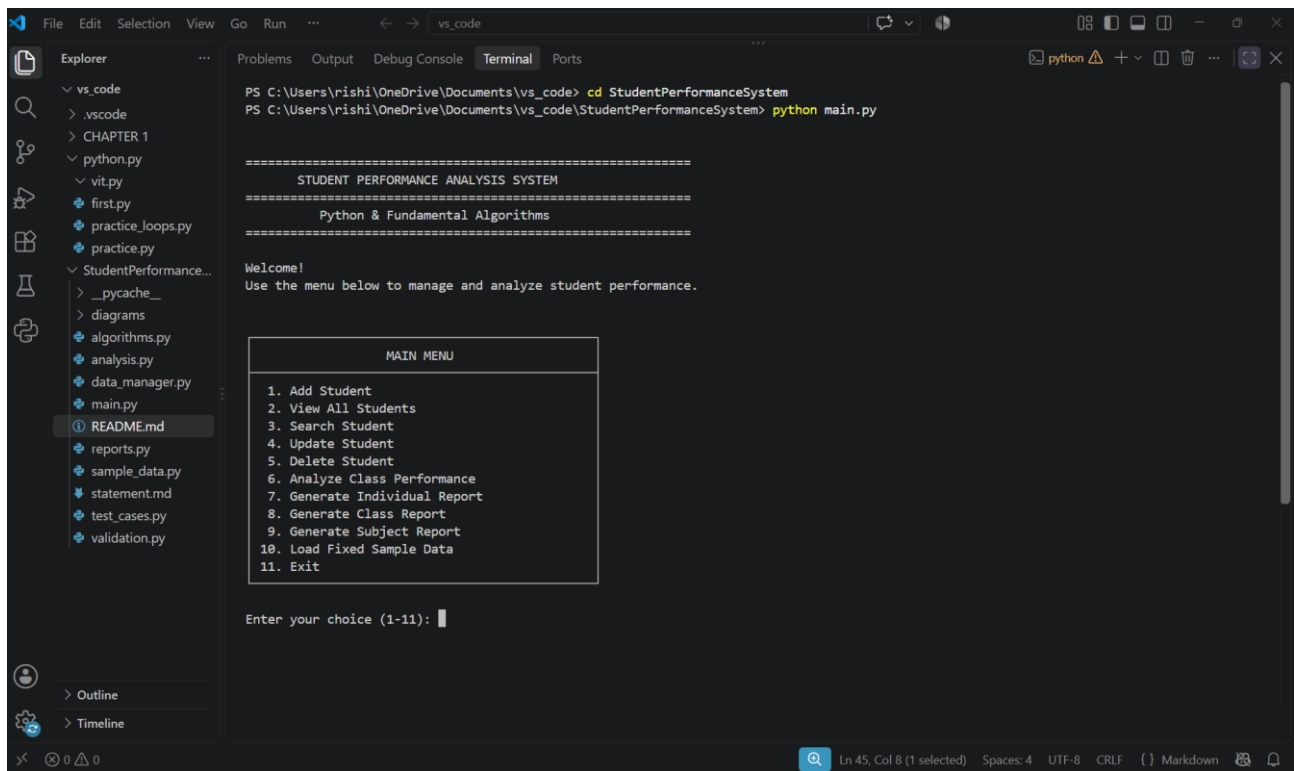
9.2 Fundamental Algorithms Used

- Summation and average calculation
- Counting based on conditions
- Maximum and minimum finding
- Linear search for student IDs
- Duplicate removal
- Partitioning students based on a condition

10. Screenshots and Results

Insert the four screenshots of the working program in the spaces below. The screenshots should be clear and readable.

Figure 1 – Main Menu



The image shows a Visual Studio Code (VS Code) interface with a terminal window open. The terminal displays the output of running a Python script named `main.py`. The output shows the program's title, a welcome message, and a main menu with 11 options. The menu is displayed in a box, and the prompt "Enter your choice (1-11):" is visible at the bottom of the terminal.

```
PS C:\Users\rishi\OneDrive\Documents\vs_code> cd StudentPerformanceSystem
PS C:\Users\rishi\OneDrive\Documents\vs_code\StudentPerformanceSystem> python main.py

=====
STUDENT PERFORMANCE ANALYSIS SYSTEM
=====
Python & Fundamental Algorithms
=====

Welcome!
Use the menu below to manage and analyze student performance.

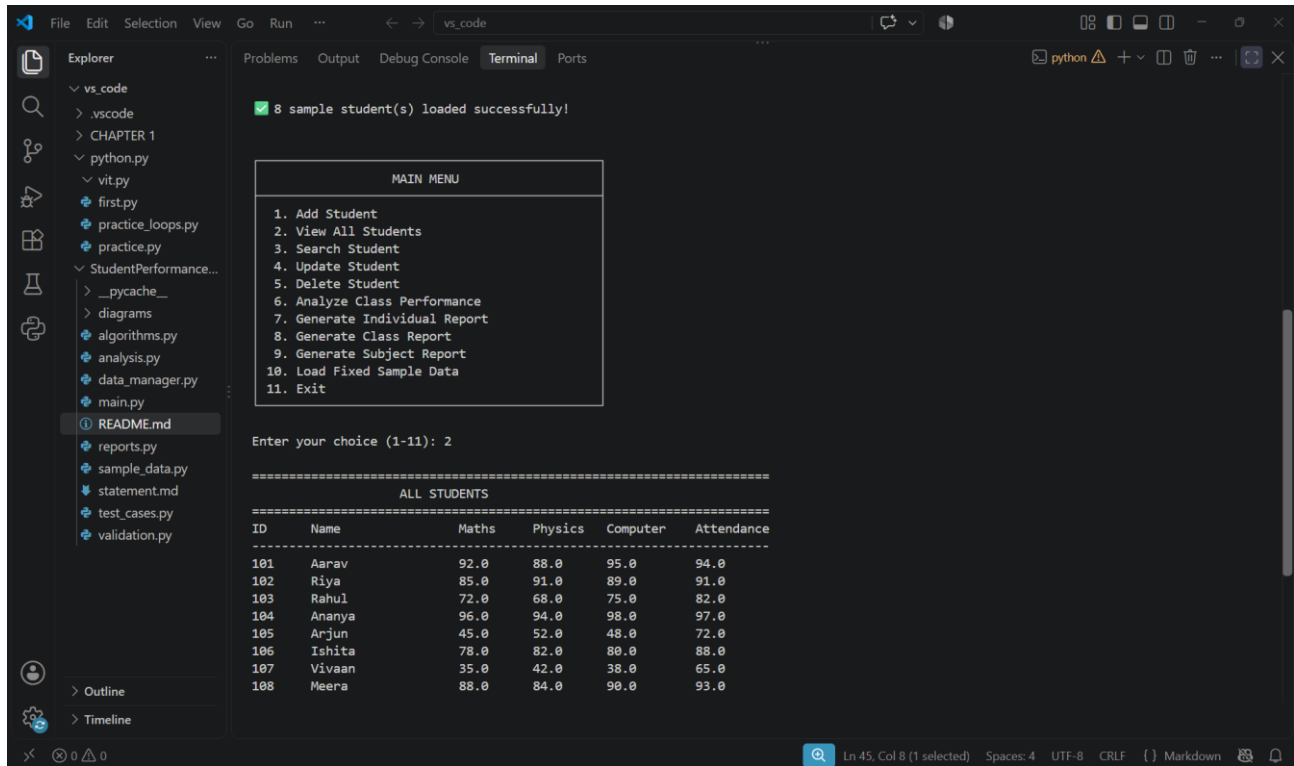
MAIN MENU

1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Analyze Class Performance
7. Generate Individual Report
8. Generate Class Report
9. Generate Subject Report
10. Load Fixed Sample Data
11. Exit

Enter your choice (1-11):
```

The VS Code interface includes a sidebar on the left with the Explorer view showing the project structure. The Explorer view shows a folder named `vs_code` containing a `.vscode` folder, a `CHAPTER 1` folder, a `python.py` file, a `vit.py` file, a `practice_loops.py` file, a `practice.py` file, a `StudentPerformance...` folder, a `__pycache__` folder, a `diagrams` folder, a `algorithms.py` file, an `analysis.py` file, a `data_manager.py` file, a `main.py` file, a `README.md` file, a `reports.py` file, a `sample_data.py` file, a `statement.md` file, a `test_cases.py` file, and a `validation.py` file. The `README.md` file is selected. The bottom status bar shows the current line and column (Ln 45, Col 8), the number of spaces (4), the encoding (UTF-8), the line ending (CRLF), and the file type (Markdown).

Figure 2 – Student Records with Fixed Sample Data

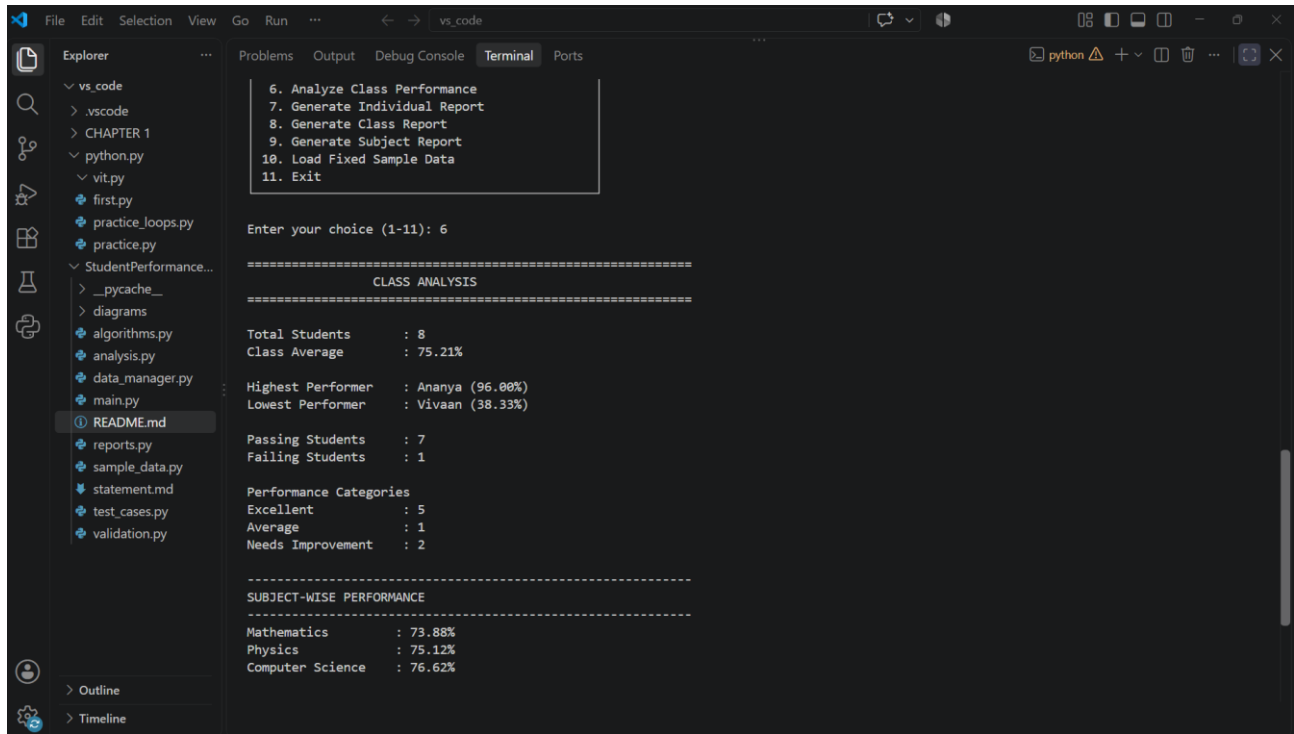


```
File Edit Selection View Go Run ...  
vs_code  
python  
8 sample student(s) loaded successfully!  
  
MAIN MENU  
1. Add Student  
2. View All Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Analyze Class Performance  
7. Generate Individual Report  
8. Generate Class Report  
9. Generate Subject Report  
10. Load Fixed Sample Data  
11. Exit  
  
Enter your choice (1-11): 2  
  
=====  
ALL STUDENTS  
=====
```

ID	Name	Maths	Physics	Computer	Attendance
101	Aanav	92.0	88.0	95.0	94.0
102	Riya	85.0	91.0	89.0	91.0
103	Rahul	72.0	68.0	75.0	82.0
104	Ananya	96.0	94.0	98.0	97.0
105	Arjun	45.0	52.0	48.0	72.0
106	Ishita	78.0	82.0	80.0	88.0
107	Vivaan	35.0	42.0	38.0	65.0
108	Meera	88.0	84.0	90.0	93.0

Ln 45, Col 8 (1 selected) Spaces: 4 UTF-8 CRLF {} Markdown

Figure 3 – Class Performance Analysis



```
File Edit Selection View Go Run ... < -> vs_code
Explorer
  vs_code
  .vscode
  CHAPTER 1
  python.py
  vit.py
  first.py
  practice_loops.py
  practice.py
  StudentPerformance...
  _pycache_
  diagrams
  algorithms.py
  analysis.py
  data_manager.py
  main.py
  README.md
  reports.py
  sample_data.py
  statement.md
  test_cases.py
  validation.py
  Outline
  Timeline
Problems Output Debug Console Terminal Ports
6. Analyze Class Performance
7. Generate Individual Report
8. Generate Class Report
9. Generate Subject Report
10. Load Fixed Sample Data
11. Exit

Enter your choice (1-11): 6

=====
CLASS ANALYSIS
=====

Total Students      : 8
Class Average       : 75.21%

Highest Performer   : Ananya (96.00%)
Lowest Performer    : Vivaan (38.33%)

Passing Students    : 7
Failing Students    : 1

Performance Categories
Excellent           : 5
Average             : 1
Needs Improvement   : 2

-----
SUBJECT-WISE PERFORMANCE
-----

Mathematics         : 73.88%
Physics              : 75.12%
Computer Science    : 76.62%
```

Figure 4 – Automated Test Results

The screenshot shows the Visual Studio Code interface with a terminal window open. The Explorer sidebar on the left shows a project structure with files like `python.py`, `vit.py`, `first.py`, `practice_loops.py`, `practice.py`, `StudentPerformance...`, `__pycache__`, `diagrams`, `algorithms.py`, `analysis.py`, `data_manager.py`, `main.py`, `README.md`, `reports.py`, `sample_data.py`, `statement.md`, `test_cases.py`, and `validation.py`. The terminal window displays the following output:

```
4. Update Student
5. Delete Student
6. Analyze Class Performance
7. Generate Individual Report
8. Generate Class Report
9. Generate Subject Report
10. Load Fixed Sample Data
11. Exit

Enter your choice (1-11): 11

=====
Thank you for using the Student Performance
Analysis System!
=====
Program terminated successfully.
=====
PS C:\Users\rishi\OneDrive\Documents\vs_code\StudentPerformanceSystem> python test_cases.py

=====
PROGRAM VERIFICATION
=====
✓ Test 1 Passed: Summation
✓ Test 2 Passed: Average
✓ Test 3 Passed: Maximum
✓ Test 4 Passed: Minimum
✓ Test 5 Passed: Duplicate Removal
✓ Test 6 Passed: Grade Calculation
✓ Test 7 Passed: Linear Search
=====
ALL TESTS PASSED SUCCESSFULLY!
=====
PS C:\Users\rishi\OneDrive\Documents\vs_code\StudentPerformanceSystem>
```

10.1 Sample Dataset

ID	Name	Mathematics	Physics	Computer Science	Attendance
101	Aarav	92	88	95	94
102	Riya	85	91	89	91
103	Rahul	72	68	75	82
104	Ananya	96	94	98	97
105	Arjun	45	52	48	72
106	Ishita	78	82	80	88
107	Vivaan	35	42	38	65
108	Meera	88	84	90	93

10.2 Expected Analysis from the Fixed Dataset

- Total students: 8
- Class average: approximately 75.21%
- Highest performer by average: Ananya (96.00%)
- Lowest performer by average: Vivaan (approximately 38.33%)
- Passing students: 7
- Failing students: 1
- Subject averages: Mathematics 73.88, Physics 75.13, Computer Science 76.63

- Excellent category (average ≥ 80): Aarav, Riya, Ananya, Ishita, and Meera
- Needs improvement category (average < 50): Arjun and Vivaan

11. Testing Approach

The project includes a dedicated `test_cases.py` module for program verification. The tests use known input values and compare the returned results with expected values.

Test	Purpose	Result
Summation	Checks the summation algorithm.	Passed
Average	Checks average calculation.	Passed
Maximum	Checks maximum-value finding.	Passed
Minimum	Checks minimum-value finding.	Passed
Duplicate Removal	Checks duplicate removal.	Passed
Grade Calculation	Checks grade boundaries.	Passed
Linear Search	Checks student lookup by ID.	Passed

The test program is executed from the terminal using: `python test_cases.py`

12. Challenges Faced

- Dividing the project into modules while keeping the responsibilities of each module clear.
- Designing input validation so that invalid values do not interrupt program execution.
- Maintaining consistent student data across CRUD operations and reports.
- Implementing fundamental algorithms explicitly instead of depending entirely on built-in functions.
- Testing different parts of the system and ensuring that the program remains executable from the command line.

13. Learnings and Key Takeaways

- Applied top-down design to break a real-world problem into smaller modules.
- Improved understanding of Python functions, modules, loops, and conditional statements.
- Practiced lists, dictionaries, and sets in a practical application.
- Implemented and applied fundamental algorithms to student data.
- Learned the importance of input validation and program verification.
- Gained experience using GitHub to organize and publish a programming project.

14. Future Enhancements

- Add persistent storage using a database or structured data file.
- Add a graphical user interface or web interface.
- Add authentication for teachers or administrators.
- Provide data visualization such as performance charts.
- Add filtering and sorting options for larger datasets.
- Support exporting reports to PDF or spreadsheet formats.
- Add more detailed attendance and semester-wise analysis.

15. Conclusion

The Student Performance Analysis System demonstrates how fundamental programming concepts and algorithms can be combined to solve a practical problem. The application provides student record management, performance analysis, report generation, input validation, and automated testing through a modular command-line interface.

The project also demonstrates the practical use of top-down design, Python data structures, functions, modules, and fundamental algorithms. The fixed sample dataset and test cases make the application suitable for repeatable demonstration and verification.

16. References

- Python 3 Documentation – <https://docs.python.org/3/>
- VITYarthi Project Instructions – Build Your Own Project instruction document.
- Course material for Introduction to Problem Solving and Programming (CSE1021).